

qPOTS: A Python Package for Sample-efficient Batch Constrained Multiobjective Bayesian Optimization

Ashwin Renganathan^{1,2}, Peter E. Bachman¹, and Kade E. Carlson¹

¹ Department of Aerospace Engineering, The Pennsylvania State University, United States ² Institute for Computational and Data Sciences, The Pennsylvania State University, United States

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: Fabian Scheipl

Reviewers:

- [@jbussemaker](#)
- [@gialmisi](#)

Submitted: 14 June 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

Designing engineered systems involve optimizing the system for some performance objective under several design constraints. In practice, though, there are typically more than one objective to meet and the objectives may conflict with each other. For instance, an aircraft designer may need to reduce the aerodynamic drag on the aircraft while ensuring passenger safety. Aerodynamic efficiency typically leads to slender aircraft cross-sections which can, however, compromise the safety of the aircraft under gusty conditions. Similar tradeoffs occur in chemical process optimization, materials discovery, and machine learning hyperparameter optimization. Optimizing simultaneously for more than one objective is called multiobjective optimization, and the solution to such problems is no longer a single point but a set of points that offer a pragmatic compromise between the objectives.

qPOTS is a Python package that solves constrained multiobjective optimization when there is no derivative information, and when every query of the objectives and constraints is expensive and thus necessitating “sample efficiency” (i.e., solving the problem with as few evaluations of objectives and constraints as possible). qPOTS is a Bayesian optimization (Frazier, 2018, p. garnett2023bayesian) approach, and implements the batch Pareto optimal Thompson sampling (qPOTS) method (Renganathan & Carlson, 2025; Thompson, 1933). The package also includes qPOTS-Decoupled, which supports decoupled objective and constraint evaluations – that is when objective and constraints are evaluated by running computer models that can be evaluated independently, leveraging independent resources, the algorithm achieves further sample efficiency by evaluating only a subset of objectives and constraints during each design iteration (Renganathan et al., 2026).

qPOTS is released under the GNU General Public License v3.0 and is built on widely used scientific Python libraries including PyTorch, GPyTorch, BoTorch, and Pymoo (Balandat et al., 2020; Blank & Deb, 2020; Gardner et al., 2018; Paszke et al., 2019).

Statement of need

The solution to a multiobjective optimization problem is called the “Pareto set”, and their image in the objective space is called the “Pareto frontier”. Computing the Pareto set/frontier is nontrivial. When derivative information is available and the objectives and constraints are smooth and not expensive to evaluate, one may use gradient-based optimization techniques over several scalarized versions of the objectives Fliege et al. (2009). However, in real-world problems, good quality derivatives are rarely available, objective/constraint smoothness is not guaranteed, and the Pareto frontier may have a complex topology (disconnected, nonconvex, etc.). In such situations, multiobjective Bayesian optimization (MOBO) serves as a suitable alternative. MOBO learns global surrogate models for the objective and constraints, and leverages Bayesian decision theory to generate a sequence of iterates that balance exploration

42 and exploitation of the design space. This involves constructing an “acquisition” function that
43 quantifies the utility of a candidate iterate, and solving an “inner” optimization problem to
44 determine the iterates. This inner optimization depends only on the surrogate model and is
45 independent of the expensive objectives and constraints.

46 MOBO is an active area of research; there are several existing works that innovate on the
47 Bayesian decision-theoretic acquisition function Daulton et al. (2020), or use scalarization to fit
48 into (single objective) BO (Knowles, 2006), or modify existing approaches to promote diversity
49 in the iterates (Konakovic Lukovic et al., 2020), among others. However, existing methods have
50 one or more of the following limitations. (1) The inner acquisition function optimization is still
51 nontrivial due to non-convexity and/or stochasticity - this ultimately impacts the accuracy of
52 the predicted Pareto set/frontier. (2) They don’t support batch sampling (generating a batch
53 of iterates). (3) They cannot handle nonlinear constraints. See (Renganathan & Carlson, 2025)
54 for further details. qPOTS was introduced to address all of these limitations, with improved
55 sample efficiency and solution accuracy being our overarching goal.

56 Evolutionary algorithms (Deb et al., 2002) are quite popular for derivative-free multiobjective
57 optimization. However, they rely on generating large populations of the objectives that must
58 be mutated and crossed-over multiple times. This, although leads to accurate solutions, can
59 penalize sample efficiency. qPOTS innovatively combines the sample efficiency of BO with the
60 accuracy of evolutionary algorithms. By fitting Gaussian process (Rasmussen & Williams,
61 2006) surrogates for the objectives and constraints, qPOTS replaces the conventional acquisition
62 function optimization in MOBO with a cheap inner multiobjective optimization over the GP
63 posterior sample paths, leveraging evolutionary algorithms. Nonlinear constraints are seamlessly
64 handled as part of the Gaussian process Thompson sampling (Renganathan & Carlson, 2025).

65 It should be mentioned that Thompson sampling based approaches, similar to qPOTS, already
66 exist for MOBO – TS-EMO and TS-TCH Paria et al. (2020). The crucial difference between qPOTS
67 and TS-EMO is that TS-EMO generates GP posterior Thompson samples, but still has to compute
68 the maximum hypervolume improvement over them to choose the candidate – this inner
69 optimization problem can be difficult to solve (Renganathan & Carlson, 2025). Additionally
70 TS-EMO is unable to handle constraints. TS-TCH scalarizes the Thompson samples to leverage
71 single-objective acquisition functions. This inherits the limitations scalarized approaches in
72 computing disconnected and nonconvex Pareto frontiers.

73 A further opportunity arises when the objectives and constraints can be evaluated asynchronously
74 using independent compute resources. For example, in aerodynamic design, objectives may be
75 aircraft performance at different operating conditions that may be evaluated separately. In
76 such situations, if the objectives and constraints are correlated with each other, MOBO can
77 proceed with only a subset of objective and constraint evaluation at each iterate. The qPOTS
78 library includes qPOTS-Decoupled (Renganathan et al., 2026) to address this, which combines
79 the qPOTS posterior sampling workflow with a multitask Gaussian process (MTGP) (Bonilla
80 et al., 2007) that jointly learns all objectives and constraints. Crucially, MTGPs learns the
81 correlations across the objectives and constraints which can be exploited to determine which
82 iterates warrant a decoupled evaluation. It should be noted that decoupled MOBO methods
83 already exist - PESMOC and HVKG can progress with partial information (Daulton et al.,
84 2023; Garrido-Merchán & Hernández-Lobato, 2019). However, existing methods depend on
85 evaluation cost difference across the objectives and constraints to determine decoupling. On
86 the other hand, qPOTS-Decoupled can decouple evaluations under uniform evaluation costs;
87 crucially, evaluation cost difference will further improve sample efficiency of qPOTS-Decoupled.

88 qPOTS builds on GPyTorch (Gardner et al., 2018) and existing GP and MTGP wrappers in
89 BoTorch (Balandat et al., 2020) for surrogate modeling, Pymoo (Blank & Deb, 2020) for
90 evolutionary algorithms, and fully supports GPU acceleration. Even though qPOTS builds on
91 these existing libraries, a need for a standalone library is compelling for the following reasons.
92 First, qPOTS is anchored on Pareto optimal Thompson sampling – this makes it compatible with
93 probabilistic models beyond GPs such as Bayesian neural networks and variational autoencoders.

Therefore, a future direction independent of GP models is possible. Second, generative models may be integrated into the framework to learn the distribution of the Pareto set $\mathcal{X}^*$ in Thompson sampling. Third, qPOTS-Decoupled can be potentially applied to other problems such as optimal experiment design and sensor placement, where optimal selection of a subset from a correlated set of tasks is of interest. Therefore, a standalone library will promote wider usage, contributions, and expansions that might not be possible when integrated into another library.

The target audience is researchers and engineers who need a sample-efficient derivative-free multiobjective optimization software in Python. qPOTS is designed for method developers benchmarking against standard acquisition strategies, and for application researchers optimizing expensive custom black-box functions under limited evaluation budgets.

State of the field

While multiobjective optimization can be solved via a variety of approaches including gradient-based approaches and evolutionary algorithms, we only review libraries relevant to MOBO here. BoTorch provides a PyTorch-based platform for Bayesian optimization (including MOBO) and already implements MOBO acquisition functions such as qNEHVI, qNParEGO, and their logarithmic variants (Ament et al., 2023; Balandat et al., 2020; Daulton et al., 2020), entropy-search acquisitions (PESMOC, MESMO, and JESMO) (Belakaria et al., 2019; Garrido-Merchán & Hernández-Lobato, 2019; Tu et al., 2022), and hypervolume knowledge gradient (Daulton et al., 2023). Trieste provides a modular TensorFlow-based Bayesian-optimization loop with multiobjective and constrained workflows (Picheny et al., 2023). TS-EMO (Bradford et al., 2018) is another Thompson sampling based MOBO algorithm (similar to qPOTS) that is based on MATLAB. qPOTS has a clear methodological distinction from these libraries as explained in the previous section and in more detail in (Renganathan & Carlson, 2025), while also having a demonstrated better empirical performance in comparison (Renganathan & Carlson, 2025).

The ability of qPOTS to go beyond GP models (as is standard in MOBO), to interface with generative modeling libraries, and extend to other applications including optimal experiment design and sensor placement, makes it attractive as a standalone package that users can build on.

Software design

qPOTS is an algorithm-specific coordination layer, not another BO library, evolutionary optimizer, or workflow engine. Each iteration must keep physical coordinates, normalized tensors, posterior state, Pymoo populations, and, in decoupled runs, task identities consistent. The package centralizes the invariant fit $\rightarrow$ posterior-path search $\rightarrow$ batch/task selection $\rightarrow$ evaluation $\rightarrow$ update transition in a runner.

The runner exposes model and acquisition objects and iteration callbacks. These seams support surrogate ablations, alternative candidate policies, and external simulator hooks without copying the optimization loop. Their cost is indirection and a larger API surface; concrete defaults, validated configuration, typed results, and fixed control flow contain it. Routine use calls `run()`, while `step()` exposes each iteration to external orchestration.

Coupled and decoupled modes can be run with MTGPs and share one state representation: task identifiers record requested outputs and unevaluated outputs become missing observations. This avoids parallel implementations, but requires stricter validation and a missing-data capable surrogate. It also makes qPOTS more than a BoTorch acquisition function: the package coordinates posterior path construction, Pareto set search, batch and output selection, and partial observation updates across library boundaries.

BoTorch and GPyTorch supply tensor-native modeling, posterior sampling, and baseline

acquisition components (Balandat et al., 2020; Gardner et al., 2018). Reimplementing them would add numerical maintenance without distinguishing qPOTS. Pymoo is retained because its vectorized bounded-problem interface, seeded execution, nondominated-set result, and callbacks fit the inner multiobjective optimization needs well (Blank & Deb, 2020). jMetalPy, pagmo/pygmo, and DEAP are credible alternatives (Benítez-Hidalgo et al., 2019; Biscani & Izzo, 2020; Fortin et al., 2012). The cost is an extra dependency and a NumPy–PyTorch/CPU boundary; isolating the adapter localizes replacement. We did not use a file/DAG orchestrator such as Snakemake (Köster & Rahmann, 2012) because each next design – and sometimes output task – depends on a newly fitted in-memory posterior. The corresponding cost is that qPOTSRunner does not provide file-based checkpointing, scheduling, or provenance; external systems can invoke `step()` and consume callbacks.

Research impact statement

qPOTS is the reference implementation for the AISTATS 2025 paper introducing Pareto optimal Thompson sampling for batch multiobjective Bayesian optimization (Renganathan & Carlson, 2025). That paper evaluates the method on synthetic and real-world benchmarks against analytical, entropy-search, Thompson-sampling, and random baselines. The software has also supported an aerodynamic design study of the NASA Common Research Model with open reproducibility materials (Carlson & Renganathan, 2026). These are demonstrated uses of the method and its research code lineage.

Independent evaluation is beginning to emerge: SPREAD includes qPOTS in a numerical comparison of MOBO methods (Hoteegni & Peitz, 2025). Other publications cite qPOTS while discussing generative molecular design, sustainable process systems, class-imbalance learning, and many-objective optimization (Jiang et al., 2026; Kudva et al., 2026, Ch. 3; Muthyala et al., 2026; Wang et al., 2025). We treat those citations as evidence of both awareness of qPOTS and as a state-of-the-art benchmark in the MOBO community.

AI usage disclosure

Anthropic Claude assisted with code refactoring and README structure before submission. OpenAI Codex assisted during review revisions with drafting tests, examples, and documentation. Generative AI was not used to design the qPOTS or qPOTS-Decoupled algorithms or to generate benchmark results. The authors reviewed every change, reran the software tests and documented examples, checked citations against publication records, and remain responsible for the manuscript and software.

Acknowledgements

The authors acknowledge the Penn State Institute for Computational and Data Sciences for access to computational research infrastructure through the Roar Core Facility.

References

- Ament, S., Daulton, S., Eriksson, D., Balandat, M., & Bakshy, E. (2023). Unexpected improvements to expected improvement for bayesian optimization. *Advances in Neural Information Processing Systems*, 36, 20577–20612. <https://doi.org/10.52202/075280-0904>
- Balandat, M., Karrer, B., Jiang, D., Daulton, S., Letham, B., Wilson, A. G., & Bakshy, E. (2020). BoTorch: A framework for efficient monte-carlo bayesian optimization. *Advances in Neural Information Processing Systems*, 33, 21524–21538. <https://proceedings.neurips.cc/paper/2020/hash/f5b1b89d98b7286673128a5fb112cb9a-Abstract.html>

- 184 Belakaria, S., Deshwal, A., & Doppa, J. R. (2019). Max-value entropy search for multi-objective
185 bayesian optimization. *Advances in Neural Information Processing Systems*, 32. [https://](https://proceedings.neurips.cc/paper_files/paper/2019/hash/82edc5c9e21035674d481640448049f3-Abstract.html)
186 [proceedings.neurips.cc/paper_files/paper/2019/hash/82edc5c9e21035674d481640448049f3-](https://proceedings.neurips.cc/paper_files/paper/2019/hash/82edc5c9e21035674d481640448049f3-Abstract.html)
187 [Abstract.html](https://proceedings.neurips.cc/paper_files/paper/2019/hash/82edc5c9e21035674d481640448049f3-Abstract.html)
- 188 Benítez-Hidalgo, A., Nebro, A. J., García-Nieto, J., Oregi, I., & Del Ser, J. (2019). jMetalPy:
189 A python framework for multi-objective optimization with metaheuristics. *Swarm and*
190 *Evolutionary Computation*, 51, 100598. <https://doi.org/10.1016/j.swevo.2019.100598>
- 191 Biscani, F., & Izzo, D. (2020). A parallel global multiobjective framework for optimization:
192 pagmo. *Journal of Open Source Software*, 5(53), 2338. [https://doi.org/10.21105/joss.](https://doi.org/10.21105/joss.02338)
193 [02338](https://doi.org/10.21105/joss.02338)
- 194 Blank, J., & Deb, K. (2020). pymoo: Multi-objective optimization in python. *IEEE Access*, 8,
195 89497–89509. <https://doi.org/10.1109/ACCESS.2020.2990567>
- 196 Bonilla, E. V., Chai, K. M. A., & Williams, C. K. I. (2007). Multi-task gaussian process
197 prediction. *Advances in Neural Information Processing Systems*, 20. [https://proceedings.](https://proceedings.neurips.cc/paper/2007/hash/66368270ffd51418ec58bd793f2d9b1b-Abstract.html)
198 [neurips.cc/paper/2007/hash/66368270ffd51418ec58bd793f2d9b1b-Abstract.html](https://proceedings.neurips.cc/paper/2007/hash/66368270ffd51418ec58bd793f2d9b1b-Abstract.html)
- 199 Bradford, E., Schweidtmann, A. M., & Lapkin, A. (2018). Efficient multiobjective optimization
200 employing gaussian processes, spectral sampling and a genetic algorithm. *Journal of Global*
201 *Optimization*, 71(2), 407–438. <https://doi.org/10.1007/s10898-018-0609-2>
- 202 Carlson, K., & Renganathan, A. (2026). Multiobjective aerodynamic design optimization of
203 the NASA common research model. *Aerospace Science and Technology*, 168, 111120.
204 <https://doi.org/10.1016/j.ast.2025.111120>
- 205 Daulton, S., Balandat, M., & Bakshy, E. (2023). Hypervolume knowledge gradient: A
206 lookahead approach for multi-objective bayesian optimization with partial information.
207 *Proceedings of the 40th International Conference on Machine Learning*, 202, 7167–7204.
208 <https://proceedings.mlr.press/v202/daulton23a.html>
- 209 Daulton, S., Balandat, M., & Bakshy, E. (2020). Differentiable expected hypervolume
210 improvement for parallel multi-objective bayesian optimization. *Advances in Neural*
211 *Information Processing Systems*, 33, 9851–9864.
- 212 Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). A fast and elitist multiobjective
213 genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2),
214 182–197. <https://doi.org/10.1109/4235.996017>
- 215 Fliege, J., Graña Drummond, L. M., & Svaiter, B. F. (2009). Newton's method for
216 multiobjective optimization. *SIAM Journal on Optimization*, 20(2), 602–626. <https://doi.org/10.1137/08071692X>
217 <https://doi.org/10.1137/08071692X>
- 218 Fortin, F.-A., De Rainville, F.-M., Gardner, M.-A., Parizeau, M., & Gagné, C. (2012). DEAP:
219 Evolutionary algorithms made easy. *Journal of Machine Learning Research*, 13, 2171–2175.
220 <https://jmlr.org/papers/v13/fortin12a.html>
- 221 Frazier, P. I. (2018). A tutorial on bayesian optimization. *arXiv Preprint arXiv:1807.02811*.
- 222 Gardner, J., Pleiss, G., Weinberger, K. Q., Bindel, D., & Wilson, A. G. (2018). GPyTorch:
223 Blackbox matrix-matrix gaussian process inference with GPU acceleration. *Advances in*
224 *Neural Information Processing Systems*, 31, 7576–7586. [https://proceedings.neurips.cc/](https://proceedings.neurips.cc/paper_files/paper/2018/hash/27e8e17134dd7083b050476733207ea1-Abstract.html)
225 [paper_files/paper/2018/hash/27e8e17134dd7083b050476733207ea1-Abstract.html](https://proceedings.neurips.cc/paper_files/paper/2018/hash/27e8e17134dd7083b050476733207ea1-Abstract.html)
- 226 Garrido-Merchán, E. C., & Hernández-Lobato, D. (2019). Predictive entropy search for
227 multi-objective bayesian optimization with constraints. *Neurocomputing*, 361, 50–68.
228 <https://doi.org/https://doi.org/10.1016/j.neucom.2019.06.025>
- 229 Hotegni, S. S., & Peitz, S. (2025). SPREAD: Sampling-based pareto front refinement via
230 efficient adaptive diffusion. *arXiv Preprint arXiv:2509.21058*. <https://doi.org/10.48550/>

- 231 [arXiv.2509.21058](#)
- 232 Jiang, C., Huang, J., & Li, M. (2026). Do we really need to approach the entire pareto
233 front in many-objective bayesian optimisation? *arXiv Preprint arXiv:2604.09417*. <https://doi.org/10.48550/arXiv.2604.09417>
- 234
- 235 Knowles, J. (2006). ParEGO: A hybrid algorithm with on-line landscape approximation
236 for expensive multiobjective optimization problems. *IEEE Transactions on Evolutionary*
237 *Computation*, 10(1), 50–66.
- 238 Konakovic Lukovic, M., Tian, Y., & Matusik, W. (2020). Diversity-guided multi-objective
239 bayesian optimization with batch evaluations. *Advances in Neural Information Processing*
240 *Systems*, 33, 17708–17720.
- 241 Köster, J., & Rahmann, S. (2012). Snakemake—a scalable bioinformatics workflow engine.
242 *Bioinformatics*, 28(19), 2520–2522. <https://doi.org/10.1093/bioinformatics/bts480>
- 243 Kudva, A., Tang, W.-T., & Paulson, J. A. (2026). Multi-objective bayesian optimization for
244 networked black-box systems: A path to greener profits and smarter designs. In C. Li
245 (Ed.), *Optimization of sustainable process systems: Multiscale models and uncertainties*
246 (pp. 63–89). John Wiley & Sons. <https://doi.org/10.1002/9781394205646.ch3>
- 247 Muthyala, M. R., Sorourifar, F., Tan, T., Peng, Y., & Paulson, J. A. (2026). Generative
248 multiobjective bayesian optimization with scalable batch evaluations for sample-efficient
249 de novo molecular design. *Industrial & Engineering Chemistry Research*, 65(1), 628–642.
250 <https://doi.org/10.1021/acs.iecr.5c03166>
- 251 Paria, B., Kandasamy, K., & Póczos, B. (2020). A flexible framework for multi-objective
252 bayesian optimization using random scalarizations. *Uncertainty in Artificial Intelligence*,
253 766–776.
- 254 Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z.,
255 Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M.,
256 Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... Chintala, S. (2019). PyTorch: An
257 imperative style, high-performance deep learning library. *Advances in Neural Information*
258 *Processing Systems*, 32, 8024–8035. <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html>
- 259
- 260 Picheny, V., Berkeley, J., Moss, H. B., Stojic, H., Granta, U., Ober, S. W., Artemev, A., Ghani,
261 K., Goodall, A., Paleyes, A., Vakili, S., Pascual-Diaz, S., Markou, S., Qing, J., Loka, N.
262 R. B. S., & Couckuyt, I. (2023). Trieste: Efficiently exploring the depths of black-box
263 functions with TensorFlow. *arXiv Preprint arXiv:2302.08436*. <https://doi.org/10.48550/arXiv.2302.08436>
- 264
- 265 Rasmussen, C. E., & Williams, C. K. I. (2006). *Gaussian processes for machine learning*. MIT
266 Press. ISBN: 978-0-262-18253-9
- 267 Renganathan, A., & Carlson, K. (2025). qPOTS: Efficient batch multiobjective bayesian
268 optimization via pareto optimal thompson sampling. In Y. Li, S. Mandt, S. Agrawal, &
269 E. Khan (Eds.), *Proceedings of the 28th international conference on artificial intelligence*
270 *and statistics* (Vol. 258, pp. 4051–4059). PMLR. <https://proceedings.mlr.press/v258/renganathan25a.html>
- 271
- 272 Renganathan, A., Carlson, K., & Bachman, P. E. (2026). *qPOTS-DOE: Batch pareto optimal*
273 *thompson sampling with decoupled oracle evaluations for constrained multiobjective bayesian*
274 *optimization*. Under review.
- 275 Thompson, W. R. (1933). On the likelihood that one unknown probability exceeds another in
276 view of the evidence of two samples. *Biometrika*, 25(3-4), 285–294. <https://doi.org/10.1093/biomet/25.3-4.285>
- 277
- 278 Tu, B., Gandy, A., Kantas, N., & Shafei, B. (2022). Joint entropy search for multi-objective

- 279 bayesian optimization. *Advances in Neural Information Processing Systems*, 35, 9922–9938.
280 <https://doi.org/https://10.52202/068431-0721>
- 281 Wang, Z., Wang, S., Ernst, D., & Xiao, C. (2025). Automated class imbalance learning via
282 few-shot multi-objective bayesian optimization with deep kernel gaussian processes. *IEEE*
283 *Access*, 13, 131839–131855. <https://doi.org/10.1109/access.2025.3591034>

DRAFT